# Lecture 16: Greedy vs Exact Search; Heuristic Framing

**Topics**

- The greedy algorithm pattern: local optimal choices
- When greedy works: optimal substructure + greedy choice property
- When greedy fails: counterexamples and proof techniques
- Heuristic algorithms: trading optimality for speed
- A* search: combining exact and heuristic approaches

**Goals**

- Recognize when greedy algorithms are correct
- Understand why Dijkstra works but other greedy algorithms fail
- Build greedy playlist construction algorithms
- Solve interval scheduling problems
- Preview A* as a framework for informed search

## Roadmap

**First half (≈45 min)**

- What makes an algorithm "greedy"?
- Greedy works: interval scheduling, Dijkstra review
- Optimal substructure and greedy choice property
- Greedy playlist construction: maximize engagement
- In-class exercise 1: Implement interval scheduling (commit required)

**Break (3 min)**

**Second half (≈45 min)**

- When greedy fails: coin change, knapsack, TSP
- Counterexamples and proof by contradiction
- Heuristic algorithms: approximate solutions
- A* search: Dijkstra + heuristic guidance
- In-class exercise 2: Greedy vs optimal comparison (commit required)
- Summary and algorithm design strategies

# Setup: Load Spotify data

We'll use tracks and plays for greedy playlist construction.

In [ ]:

```
import csv
import heapq
from collections import defaultdict, Counter
import random

# Load tracks
tracks = {}
with open('data/tracks.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        tracks[row['track_id']] = row

# Load plays
plays = []
with open('data/plays.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        plays.append(row)

print(f"Loaded {len(tracks)} tracks and {len(plays)} plays")
```

# Part 1: The Greedy Algorithm Pattern

## What is a greedy algorithm?

**Core idea:** Make the locally optimal choice at each step, hoping for a globally optimal solution.

**General pattern:**

```
def greedy_algorithm(candidates):
    solution = []
    while not done:
        # Choose best local option
        choice = select_best(candidates)
        if is_feasible(choice, solution):
            solution.append(choice)
        candidates.remove(choice)
    return solution
```

**Key insight:** Never reconsider or backtrack — each choice is final.

# Examples we've seen

**Dijkstra's algorithm (greedy, correct):**

- Local choice: Always expand the closest unvisited node
- Works because: Once we pop a node, we've found its shortest path
- Complexity: O((V + E) log V)

**Huffman coding (greedy, correct):**

- Local choice: Merge two least-frequent symbols
- Works because: Optimal prefix-free code has this structure
- Complexity: O(n log n)

**BFS/DFS traversal (greedy, depends on goal):**

- Local choice: Next neighbor in order
- Works for: Reachability, connectivity
- Doesn't optimize: Path length unless BFS on unweighted graphs

# Part 2: When Greedy Works

## Two required properties

**1. Optimal substructure**

- Optimal solution contains optimal solutions to subproblems
- Example: Shortest path A→C via B requires shortest A→B and B→C

**2. Greedy choice property**

- Can make a locally optimal choice that leads to global optimum
- Example: In Dijkstra, expanding closest node is always safe

**If both hold → greedy algorithm is correct!**

**If either fails → greedy may give suboptimal results**

# Classic example: Interval scheduling

**Problem:** You have n events with start/end times. Select maximum number of non-overlapping events.

**Example (concert scheduling):**

```
Event A: [1, 3]   (Taylor Swift)
Event B: [2, 5]   (Ed Sheeran)
Event C: [4, 7]   (Billie Eilish)
Event D: [6, 9]   (Post Malone)
```

**Greedy strategies to consider:**

1. Choose shortest event first
2. Choose event that starts earliest
3. Choose event that ends earliest ← **THIS ONE WORKS!**
4. Choose event with least conflicts

In [ ]:

```python
def interval_scheduling(events):
    """
    Select maximum number of non-overlapping events.

    Greedy strategy: Choose event that ends earliest.

    Args:
        events: list of (name, start, end) tuples

    Returns:
        list of selected event names
    """
    # Sort by end time (greedy choice)
    sorted_events = sorted(events, key=lambda e: e[2])

    selected = []
    last_end = float('-inf')

    for name, start, end in sorted_events:
        # Check if event is feasible (no overlap)
        if start >= last_end:
            selected.append(name)
            last_end = end

    return selected

# Example: Concert scheduling
concerts = [
    ('Taylor Swift', 1, 3),
    ('Ed Sheeran', 2, 5),
```

```
    ('Billie Eilish', 4, 7),
    ('Post Malone', 6, 9),
    ('Ariana Grande', 8, 10)
]

result = interval_scheduling(concerts)
print(f"Selected {len(result)} concerts: {result}")
# Output: Selected 3 concerts: ['Taylor Swift', 'Billie Eilish', 'Ariana
```

# Why ”earliest end time” works

**Proof sketch (exchange argument):**

1. Let G = greedy solution, O = some optimal solution
2. Suppose first event in G ends at time $t_1$, first in O ends at $t_2$
3. If $t_1 \leq t_2$: Can replace O's first event with G's → still optimal
4. Repeat inductively for remaining events
5. Therefore: Greedy is optimal!

**Intuition:** Finishing early leaves maximum room for future events.

**Complexity:** O(n log n) for sorting, O(n) for selection → **O(n log n) total**

# Application: Greedy playlist construction

**Problem:** Build a 60-minute playlist that maximizes engagement.

**Greedy strategy:** At each step, add the track with highest engagement per minute.

**Engagement metric:**

- Play count × completion rate
- Or: skip rate, saves, shares

**Is this optimal?** Depends on constraints!

- If tracks are independent → greedy is optimal
- If playlist flow matters (genre transitions) → greedy may fail

In [ ]:

```python
def greedy_playlist(tracks, plays, max_duration_sec):
    """
    Build playlist that maximizes engagement using greedy strategy.

    Greedy choice: Add track with highest engagement per second.

    Args:
        tracks: dict of track_id -> track info (must have 'duration_ms')
        plays: list of play records (track_id, user_id)
        max_duration_sec: maximum playlist duration in seconds

    Returns:
        (playlist_tracks, total_duration, total_engagement)
    """
    # Calculate engagement score for each track
    play_counts = Counter(p['track_id'] for p in plays)

    candidates = []
    for track_id, track in tracks.items():
        duration_sec = int(track.get('duration_ms', 180000)) / 1000
        engagement = play_counts.get(track_id, 0)

        if duration_sec > 0:
            engagement_rate = engagement / duration_sec
            candidates.append({
                'track_id': track_id,
                'name': track.get('track_name', 'Unknown'),
                'duration': duration_sec,
                'engagement': engagement,
                'rate': engagement_rate
```

```
            })

    # Sort by engagement rate (greedy choice)
    candidates.sort(key=lambda t: t['rate'], reverse=True)

    # Greedily select tracks
    playlist = []
    total_duration = 0
    total_engagement = 0

    for track in candidates:
        if total_duration + track['duration'] <= max_duration_sec:
            playlist.append(track)
            total_duration += track['duration']
            total_engagement += track['engagement']

    return (playlist, total_duration, total_engagement)

# Example: Build 30-minute playlist
playlist, duration, engagement = greedy_playlist(tracks, plays, max_dura
print(f"Playlist: {len(playlist)} tracks, {duration/60:.1f} minutes, {en
print(f"\nTop 3 tracks:")
for track in playlist[:3]:
    print(f"  {track['name']}: {track['engagement']} plays, {track['rate
```

# Exercise 1: Interval scheduling with weights (20 min)

**Problem:** You have n recording studio sessions. Each session has start/end times and a value (revenue). Select non-overlapping sessions to maximize total revenue.

**Example:**

```
Session A: [1, 3], value=100
Session B: [2, 5], value=200
Session C: [4, 7], value=150
```

**Questions:**

1. Does "earliest end time" greedy still work?
2. Does "highest value first" greedy work?
3. What about "highest value per unit time" greedy?

**Task:** Implement all three strategies and test on the example.

**Signature:**

```python
def weighted_interval_scheduling(sessions, strategy='earliest_end'):
    """
    Args:
        sessions: list of (name, start, end, value)
```

```
        strategy: 'earliest_end', 'highest_value', or 'value_per_time'

    Returns:
        (selected_sessions, total_value)
    """
```

**Commit:** `git add exercise1.py && git commit -m "Exercise 1: Weighted interval scheduling"`

In [ ]:

```python
# Exercise 1 starter code
def weighted_interval_scheduling(sessions, strategy='earliest_end'):
    """
    Select non-overlapping sessions to maximize total value.

    Args:
        sessions: list of (name, start, end, value) tuples
        strategy: 'earliest_end', 'highest_value', or 'value_per_time'

    Returns:
        (selected_sessions, total_value)
    """
    # TODO: Implement three greedy strategies
    # 1. Sort by end time
    # 2. Sort by value (descending)
    # 3. Sort by value per unit time (descending)

    return ([], 0)
```

# Exercise 1 Solution

In [ ]:

```python
def weighted_interval_scheduling(sessions, strategy='earliest_end'):
    """
    Select non-overlapping sessions to maximize total value.

    Args:
        sessions: list of (name, start, end, value) tuples
        strategy: 'earliest_end', 'highest_value', or 'value_per_time'

    Returns:
        (selected_sessions, total_value)
    """
    # Choose sorting strategy
    if strategy == 'earliest_end':
        sorted_sessions = sorted(sessions, key=lambda s: s[2])
    elif strategy == 'highest_value':
        sorted_sessions = sorted(sessions, key=lambda s: s[3], reverse=
    elif strategy == 'value_per_time':
        sorted_sessions = sorted(sessions, key=lambda s: s[3] / (s[2] -
    else:
        raise ValueError(f"Unknown strategy: {strategy}")

    # Greedy selection
    selected = []
    total_value = 0
    last_end = float('-inf')
```

```python
    for name, start, end, value in sorted_sessions:
        if start >= last_end:
            selected.append(name)
            total_value += value
            last_end = end

    return (selected, total_value)

# Test all strategies
test_sessions = [
    ('A', 1, 3, 100),
    ('B', 2, 5, 200),
    ('C', 4, 7, 150),
]

for strategy in ['earliest_end', 'highest_value', 'value_per_time']:
    selected, value = weighted_interval_scheduling(test_sessions, strate
    print(f"{strategy:20s}: {selected} → total value = {value}")

# Output:
# earliest_end         : ['A', 'C'] → total value = 250
# highest_value        : ['B'] → total value = 200
# value_per_time       : ['B'] → total value = 200

# Note: None of the greedy strategies find optimal solution ['A', 'C'] =
```

# Break (3 minutes)

Stand up, stretch, grab water. Next: When greedy fails and what to do about it.

# Part 3: When Greedy Fails

## Weighted interval scheduling (continued)

**Key insight:** Adding weights breaks the greedy choice property!

**Counterexample:**

```
A: [0, 10], value=10
B: [0,  3], value=5
C: [3,  6], value=5
D: [6, 10], value=5
```

**Greedy (earliest end):** B → C → D = 15

**Optimal:** A = 10? No! B+C+D = 15 is better.

**Wait, different counterexample:**

```
A: [0, 10], value=100
B: [0,  3], value=10
C: [3,  6], value=10
D: [6, 10], value=10
```

**Greedy (earliest end):** B → C → D = 30

**Optimal:** A = 100 ✓

**Requires:** Dynamic programming to solve optimally! (Lecture 22-23)

# Classic failure: Coin change

**Problem:** Make change for amount n using fewest coins.

**US coins:** [1, 5, 10, 25] cents

**Greedy:** Always take largest coin ≤ remaining amount

**Example 1 (greedy works):**

- Amount: 63 cents
- Greedy: 25 + 25 + 10 + 1 + 1 + 1 = 6 coins
- Optimal: Same!

**Example 2 (greedy fails):**

- Coins: [1, 3, 4]
- Amount: 6
- Greedy: 4 + 1 + 1 = 3 coins
- Optimal: 3 + 3 = 2 coins ✓

In [ ]:

```python
def greedy_coin_change(coins, amount):
    """
    Make change using greedy strategy (largest coin first).

    Args:
        coins: list of coin denominations (sorted descending)
        amount: target amount

    Returns:
        list of coins used
    """
    result = []
    remaining = amount

    for coin in sorted(coins, reverse=True):
        while remaining >= coin:
            result.append(coin)
            remaining -= coin

    return result if remaining == 0 else None

# US coins: greedy works
us_coins = [1, 5, 10, 25]
result = greedy_coin_change(us_coins, 63)
print(f"US coins for 63¢: {result} ({len(result)} coins)")
# Output: [25, 25, 10, 1, 1, 1] (6 coins)

# Pathological coins: greedy fails
bad_coins = [1, 3, 4]
result = greedy_coin_change(bad_coins, 6)
```

```
print(f"\nBad coins for 6: {result} ({len(result)} coins)")
print(f"Optimal: [3, 3] (2 coins) — greedy is suboptimal!")
# Output: [4, 1, 1] (3 coins)
```

# Classic failure: Knapsack problem

**Problem:** You have a knapsack with capacity W. Items have weight and value. Maximize value.

**Two versions:**

1. **Fractional knapsack:** Can take fractions of items → greedy works!
2. **0/1 knapsack:** Must take whole items → greedy fails!

**Example:**

```
Capacity: 10 kg
Item A: 6 kg, value 30  (5.00 per kg)
Item B: 5 kg, value 25  (5.00 per kg)
Item C: 4 kg, value 20  (5.00 per kg)
```

**Greedy (value per kg):** A + 4kg of C = 30 + 20 = 50 (fractional)

**Greedy (0/1):** A only = 30

**Optimal (0/1):** B + C = 25 + 20 = 45 ✓

# Classic failure: Traveling Salesman Problem (TSP)

**Problem:** Visit all cities exactly once, minimize total distance.

**Greedy (nearest neighbor):**

1. Start at arbitrary city
2. Always go to nearest unvisited city
3. Return to start

**Performance:** Can be 2x or worse than optimal!

**Example:**

```
Cities on a line: A -- B -- C -- D
Distances: A-B=1, B-C=1, C-D=1, A-C=2, A-D=3, B-D=2

Greedy from A: A→B→C→D→A = 1+1+1+3 = 6
Optimal: A→D→C→B→A = 3+1+1+1 = 6 (same in this case)

But with different layout, greedy can be much worse!
```

**TSP is NP-hard:** No known polynomial-time exact algorithm.

In [ ]:

```python
def greedy_tsp(cities, distances):
    """
    Solve TSP using nearest neighbor greedy heuristic.

    Args:
        cities: list of city names
        distances: dict mapping (city1, city2) -> distance

    Returns:
        (tour, total_distance)
    """
    if not cities:
        return ([], 0)

    tour = [cities[0]]
    unvisited = set(cities[1:])
    total_dist = 0

    while unvisited:
        current = tour[-1]
        # Find nearest unvisited city
        nearest = min(unvisited, key=lambda c: distances.get((current, c
        tour.append(nearest)
        total_dist += distances.get((current, nearest), 0)
        unvisited.remove(nearest)

    # Return to start
    total_dist += distances.get((tour[-1], tour[0]), 0)

    return (tour, total_dist)
```

```python

# Example: Concert tour
cities = ['New York', 'Boston', 'Philadelphia', 'Washington DC']
distances = {
    ('New York', 'Boston'): 215,
    ('New York', 'Philadelphia'): 95,
    ('New York', 'Washington DC'): 225,
    ('Boston', 'Philadelphia'): 310,
    ('Boston', 'Washington DC'): 440,
    ('Philadelphia', 'Washington DC'): 140,
}
# Make symmetric
distances.update({(b, a): d for (a, b), d in list(distances.items())})

tour, dist = greedy_tsp(cities, distances)
print(f"Greedy tour: {' → '.join(tour)} → {tour[0]}")
print(f"Total distance: {dist} miles")
```

# Part 4: Heuristic Algorithms

## When exact algorithms are too slow

**NP-hard problems:**

- No known polynomial-time exact algorithm
- Best known: exponential time (brute force, branch-and-bound)
- Examples: TSP, knapsack, SAT, graph coloring

**Heuristic approach:**

- Trade optimality for speed
- Find "good enough" solution quickly
- Often greedy-based with improvements

**Types of heuristics:**

1. **Constructive:** Build solution from scratch (greedy)
2. **Local search:** Start with solution, improve iteratively
3. **Metaheuristics:** Simulated annealing, genetic algorithms
4. **Approximation algorithms:** Provable worst-case guarantees

# Heuristic example: 2-opt for TSP

**Algorithm:**

1. Start with greedy tour (or random)
2. Try swapping pairs of edges
3. Keep swap if it improves tour
4. Repeat until no improvement

**Example swap:**

```
Before: A → B → C → D → A
Swap edges (A,B) and (C,D) with (A,C) and (B,D)
After:  A → C → B → D → A
```

**Performance:**

- Much better than pure greedy
- Still no optimality guarantee
- $O(n^2)$ per iteration, may need many iterations

In [ ]:

```python
def two_opt_tsp(cities, distances, max_iterations=100):
    """
    Improve TSP tour using 2-opt local search.

    Args:
        cities: list of city names
        distances: dict mapping (city1, city2) -> distance
        max_iterations: maximum improvement iterations

    Returns:
        (tour, total_distance)
    """
    # Start with greedy tour
    tour, best_dist = greedy_tsp(cities, distances)
    tour = tour[:-1]  # Remove duplicate start city

    improved = True
    iterations = 0

    while improved and iterations < max_iterations:
        improved = False

        # Try all pairs of edges
        for i in range(len(tour) - 1):
            for j in range(i + 2, len(tour)):
                # Current edges: (tour[i], tour[i+1]) and (tour[j], tou
                # Swap to: (tour[i], tour[j]) and (tour[i+1], tour[j+1]

                # Calculate change in distance
                a, b = tour[i], tour[i + 1]
```

```
                c, d = tour[j], tour[(j + 1) % len(tour)]

                current = distances.get((a, b), 0) + distances.get((c, d
                new = distances.get((a, c), 0) + distances.get((b, d), 0

                if new < current:
                    # Reverse segment between i+1 and j
                    tour[i + 1:j + 1] = reversed(tour[i + 1:j + 1])
                    improved = True

        iterations += 1

    # Calculate final distance
    total = sum(distances.get((tour[i], tour[(i + 1) % len(tour)]), 0)
                for i in range(len(tour)))

    return (tour, total)

# Compare greedy vs 2-opt
greedy_tour, greedy_dist = greedy_tsp(cities, distances)
opt_tour, opt_dist = two_opt_tsp(cities, distances)

print(f"Greedy: {greedy_dist} miles")
print(f"2-opt:  {opt_dist} miles")
print(f"Improvement: {greedy_dist - opt_dist} miles ({100*(greedy_dist-d
```

# Part 5: A* Search (Preview)

## Dijkstra + heuristic guidance

**Dijkstra's limitation:** Explores uniformly in all directions.

**A* idea:** Use a heuristic to guide search toward the goal.

**Formula:**

```
f(n) = g(n) + h(n)

g(n) = actual cost from start to n (like Dijkstra)
h(n) = heuristic estimate from n to goal
f(n) = estimated total cost through n
```

**Algorithm:** Like Dijkstra, but prioritize by f(n) instead of g(n).

# A* requirements

**Admissible heuristic:** h(n) ≤ true cost from n to goal

- Never overestimate remaining cost
- Example: Straight-line distance in road networks

**Consistent heuristic:** h(n) ≤ cost(n, n') + h(n') for all neighbors n'

- Stronger than admissible
- Ensures optimal solution

**If heuristic is admissible → A* finds optimal path**

**If h(n) = 0 → A* reduces to Dijkstra**

**If h(n) = true cost → A* goes straight to goal (perfect!)**

In [ ]:

```python
def astar_search(graph, start, goal, heuristic):
    """
    Find shortest path using A* search.

    Args:
        graph: dict mapping node -> [(neighbor, cost), ...]
        start: starting node
        goal: target node
        heuristic: function(node) -> estimated cost to goal

    Returns:
        (distance, path) or (None, None)
    """
    if start == goal:
        return (0, [start])

    # g(n) = actual cost from start
    g_cost = {start: 0}
    parent = {start: None}

    # Priority queue: (f(n), node) where f(n) = g(n) + h(n)
    heap = [(heuristic(start), start)]

    while heap:
        f_cost, node = heapq.heappop(heap)

        # Found goal
        if node == goal:
            path = []
            current = goal
```

```
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            return (g_cost[goal], path)

        # Skip if already processed with lower cost
        if f_cost > g_cost[node] + heuristic(node):
            continue

        # Expand neighbors
        for neighbor, edge_cost in graph.get(node, []):
            new_g = g_cost[node] + edge_cost

            if new_g < g_cost.get(neighbor, float('inf')):
                g_cost[neighbor] = new_g
                parent[neighbor] = node
                f = new_g + heuristic(neighbor)
                heapq.heappush(heap, (f, neighbor))

    return (None, None)

# Example: Artist collaboration with heuristic
# Heuristic: estimated collaboration strength to target
def collaboration_heuristic(artist, target='Rihanna'):
    """Dummy heuristic: return 0 (reduces to Dijkstra)"""
    return 0

# Would need actual collaboration data to build meaningful heuristic
```

# Exercise 2: Greedy vs optimal comparison (20 min)

**Task:** Implement and compare greedy vs brute-force optimal for a small knapsack problem.

**Problem:**

```
Capacity: 15 units
Items:
  A: weight=5,  value=10
  B: weight=4,  value=8
  C: weight=6,  value=12
  D: weight=3,  value=6
```

**Implement:**

1. Greedy by value per weight (descending)
2. Brute force (try all 2^n subsets)

**Signature:**

```python
def knapsack_greedy(items, capacity):
    """Returns (selected_items, total_value)"""

def knapsack_optimal(items, capacity):
    """Returns (selected_items, total_value)"""
```

**Commit:** `git add exercise2.py && git commit -m "Exercise 2: Greedy vs optimal knapsack"`

In [ ]:

```python
# Exercise 2 starter code
def knapsack_greedy(items, capacity):
    """
    Solve 0/1 knapsack using greedy (value per weight).

    Args:
        items: list of (name, weight, value) tuples
        capacity: maximum weight

    Returns:
        (selected_items, total_value, total_weight)
    """
    # TODO: Sort by value per weight, greedily select
    return ([], 0, 0)

def knapsack_optimal(items, capacity):
    """
    Solve 0/1 knapsack using brute force (all subsets).

    Args:
        items: list of (name, weight, value) tuples
        capacity: maximum weight

    Returns:
        (selected_items, total_value, total_weight)
    """
    # TODO: Try all 2^n subsets, track best
    return ([], 0, 0)
```

# Exercise 2 Solution

In [ ]:

```python
def knapsack_greedy(items, capacity):
    """
    Solve 0/1 knapsack using greedy (value per weight).

    Args:
        items: list of (name, weight, value) tuples
        capacity: maximum weight

    Returns:
        (selected_items, total_value, total_weight)
    """
    # Sort by value per weight (descending)
    sorted_items = sorted(items, key=lambda i: i[2] / i[1], reverse=True)

    selected = []
    total_value = 0
    total_weight = 0

    for name, weight, value in sorted_items:
        if total_weight + weight <= capacity:
            selected.append(name)
            total_value += value
            total_weight += weight

    return (selected, total_value, total_weight)
```

```python
def knapsack_optimal(items, capacity):
    """
    Solve 0/1 knapsack using brute force (all subsets).

    Args:
        items: list of (name, weight, value) tuples
        capacity: maximum weight

    Returns:
        (selected_items, total_value, total_weight)
    """
    n = len(items)
    best = ([], 0, 0)

    # Try all 2^n subsets
    for mask in range(1 << n):
        selected = []
        total_value = 0
        total_weight = 0

        for i in range(n):
            if mask & (1 << i):
                name, weight, value = items[i]
                selected.append(name)
                total_value += value
                total_weight += weight

        # Check if valid and better
        if total_weight <= capacity and total_value > best[1]:
            best = (selected, total_value, total_weight)
```

```
    return best

# Test
test_items = [
    ('A', 5, 10),
    ('B', 4, 8),
    ('C', 6, 12),
    ('D', 3, 6),
]

greedy_result = knapsack_greedy(test_items, 15)
optimal_result = knapsack_optimal(test_items, 15)

print(f"Greedy:  {greedy_result[0]} → value={greedy_result[1]}, weight=
print(f"Optimal: {optimal_result[0]} → value={optimal_result[1]}, weigh

if greedy_result[1] < optimal_result[1]:
    print(f"\nGreedy is suboptimal! Gap: {optimal_result[1] - greedy_res
```

# Summary

## Key takeaways

**Greedy algorithms:**

- Make locally optimal choices without backtracking
- Fast: usually O(n log n) due to sorting
- Work when: optimal substructure + greedy choice property

**When greedy works:**

- Dijkstra's shortest path (non-negative weights)
- Interval scheduling (unweighted)
- Huffman coding
- Fractional knapsack
- Minimum spanning tree (Prim, Kruskal)

**When greedy fails:**

- Weighted interval scheduling → need DP
- Coin change (arbitrary denominations) → need DP
- 0/1 knapsack → need DP
- TSP → NP-hard, use heuristics

**Heuristic approaches:**

- Trade optimality for speed
- Greedy + local search (2-opt)
- A* = Dijkstra + admissible heuristic
- Approximation algorithms with guarantees

# Algorithm design strategies

**How to approach a new problem:**

1. **Try greedy first**
    - Prototype in 10 minutes
    - Find counterexample or prove correct
2. **If greedy fails, consider:**
    - Dynamic programming (optimal substructure)
    - Divide and conquer (independent subproblems)
    - Backtracking/branch-and-bound (exhaustive)
    - Heuristics (good enough, fast)
3. **Prove or disprove:**
    - Exchange argument (greedy is optimal)
    - Counterexample (greedy fails)
    - Reduction to known problem (NP-hard)
4. **Production considerations:**
    - Exact vs approximate
    - Speed vs quality tradeoff
    - Problem size and constraints

# Complexity table

| Problem | Greedy? | Exact Algorithm | Complexity | Notes |
|---|---|---|---|---|
| **Interval scheduling** | ✓ Yes | Greedy (earliest end) | O(n log n) | Optimal |
| **Weighted intervals** | ✗ No | Dynamic programming | O(n log n) | Greedy fails |
| **Shortest path** | ✓ Yes | Dijkstra | O((V+E) log V) | Non-negative weights |
| **Coin change (US)** | ✓ Yes | Greedy | O(n) | Specific denominations |
| **Coin change (any)** | ✗ No | Dynamic programming | O(nW) | Greedy fails |
| **Fractional knapsack** | ✓ Yes | Greedy (value/weight) | O(n log n) | Optimal |
| **0/1 knapsack** | ✗ No | Dynamic programming | O(nW) | Greedy fails |
| **TSP** | ✗ No | Heuristic (2-opt, etc.) | $O(n^2)$ per iter | NP-hard |
| **MST** | ✓ Yes | Prim/Kruskal | O(E log V) | Optimal (L19) |

# Next lecture preview

**Lecture 17: Recurrences, Master Theorem, Runtime Models**

- Analyzing recursive algorithms
- Master theorem for divide-and-conquer
- Amortized analysis
- Python cost models in depth

**Homework:**

- Review greedy proofs (exchange arguments)
- Think about when greedy works in your projects
- Read about dynamic programming as alternative

**Project 3 (Rock Tour) due March 30**

- BFS/DFS are greedy in some sense (first valid path)
- But for shortest path on unweighted graph, BFS is optimal!